

Planar 3-Link Biped Kinetic and Potential Energies including D,C,G matrices according to Lagrange Formalism (MATLAB Code)

Ashish Kumar, Kiran Sairam Bethi Balagangadaran, Marlon Rivas, Prathyush Ravula

Coordinate Frame Convention

Inertia Coordinate Frame Convention

```
x (up)
^
|
|
0 ----> y (left)
```

Origin 0 is at the stance foot contact point.

+x axis points UPWARD.

+y axis points LEFT.

Angles are measured CLOCKWISE and taken as POSITIVE.

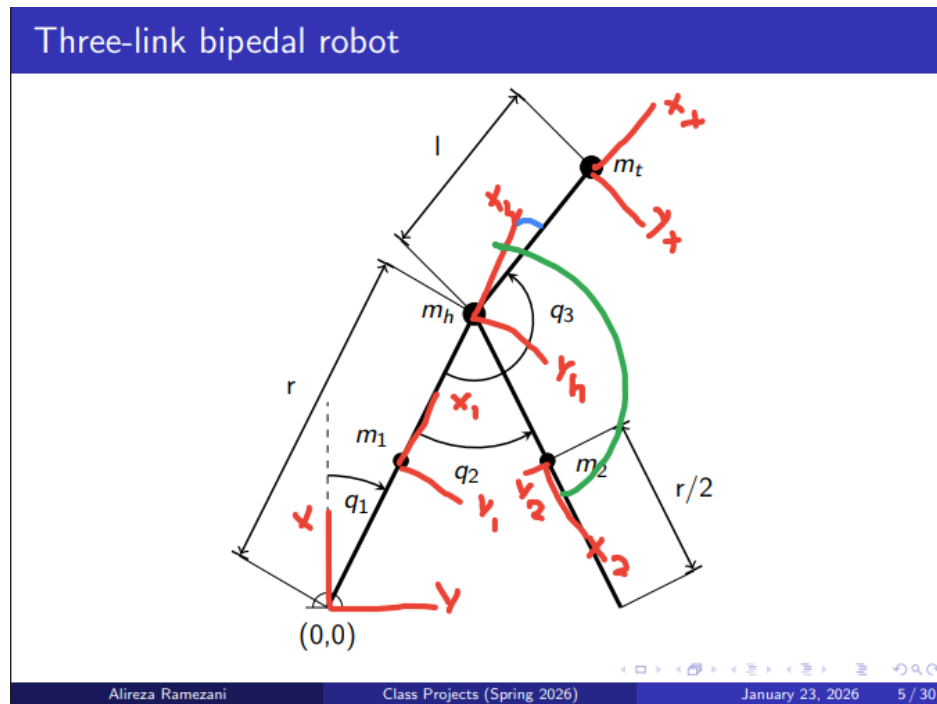


Figure 1: Coordinate frames

MATLAB Code Mini Project 3

1 Function Zero Dynamics file

```
% Simulation zero dynamics of 3 link biped in the sagittal plane
%
% Inputs:
%     t: time
%     z: cyclic variables [q1, dq1]
%     a: bezier coefficient for q2 - a(1:5), and q3 - a(6:10)
%     s_params: [z_min, z_max] - max and min angles in gait
%
% Outputs:
%     dz = [dq1, ddq1]
%
function dz = func_zero_dynamics(t,z,a,s_params)

z_min = s_params(1);
z_max = s_params(2);
delq = z_max - z_min;

q1 = z(1); dq1 = z(2);

% Computes gait timing variable s
%
% Inputs:
%     q1: cyclic variable
%     q1_min: minimum value for q1 during gait
%     q1_max: max value for gait
%
% Output:
%     s: gait timing variable
%
s = func_gait_timing(q1,z_min,z_max);

alpha2 = a(1:5);
alpha3 = a(6:10);

%Need bezier formulation
function b = control_points(s, a2)
    M = 4;
    b = 0;
    for k = 0:M
        b = b + a2(k+1) * (factorial(M)/(factorial(k)*factorial(M-k))) * s
            ^k * (1-s)^(M-k);
    end
end

b2 = control_points(s, alpha2);
b3 = control_points(s, alpha3);

h_desired = [q1; b2; b3];
```

```

% find based on s and bezier definition
q2 = b2;
%q2_desired = q_converted(2);
q3 = b3;
%q3_desired = q_converted(3);

function db_ds = compute_partial_bezier(M, alpha, s)

    db_ds = 0;
    for k = 0:M-1
        db_ds = db_ds + (alpha(k+2) - alpha(k+1)) *(factorial(M)/(
            factorial(k)*factorial(M-1-k))) *s^k * (1-s)^(M-1-k);
    end
end

%      (thousands of times per simulation step), spamming the console

M = 4;
db_ds2 = compute_partial_bezier(M,alpha2,s);
db_ds3 = compute_partial_bezier(M,alpha3,s);

dq2 = db_ds2 * (dq1/delq);
dq3 = db_ds3 * (dq1/delq);

q = [q1, q2, q3];
dq = [dq1, dq2, dq3];

% Get model paramters
[r,m,Mh,Mt,l,g] = func_model_params;
params = [r,m,Mh,Mt,l,g];

% Compute D,C,G,B matrices
% Inputs:
%     q
%     dq
%     params
%
% Output: [D,C,G,B]
%
[D,C,G,B] = func_compute_D_C_G_B(q,dq,params);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Apply dynamics partitioning to compute the zero
dynamics
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% equations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% E.g.,
D1 = D(1,1);
D2 = D(1,2:3);

H0 = [1, 0, 0];

```

```

    0, 1, 0];
c = [0,0,1];

H1 = H0;

%-----%

% Get beta1 term to compute dz
%
% Inputs: s, [dq1, delq], [alpha2, alpha3]
%         s: gait timing variable
%         dq1: velocity of cyclic variable = z2
%         delq: difference between z_max and z_min in s, needed when
%               computing db/ds
%         alpha2: Bezier coefficient (1st to 5th) for q2
%         alpha3: Bezier coefficient (1st to 5th) for q3
%
% Output:
%         beta1: one of 2 matrices obtained when computing ddq_b (
%               acceleration
%               of body coordinates)
%                $y = h(x) = q_b - b(s)$ , when  $y = 0$ ,  $q_b = b(s)$ 
%               taking two time derivatives we get:
%                $ddq_b = d/ds(db(s)/ds*ds/dt)*ds/dt + db(s)/ds * d^2(s)/dt$ 
%                $^2$ 
%                $= beta1 + beta2*ddq_1$ 
%               Note:  $d^2(s)/dt^2 = 1/delq*d^2(q1)/dt^2$ 
%
%               beta1 =  $d/ds(db(s)/ds*ds/dt)*ds/dt$ 
%               beta2 =  $db(s)/ds*1/delq$ 
%
beta1 = func_compute_beta1(s, [dq1, delq], [alpha2, alpha3]);

% Compute 1st partial derivative of Bezier polynomial
% Inputs:
%         s: gait timing variable
%         M: order of Bezier polynomial, in this case 4th order
%         alpha: Coefficients for the polynomial, need M+1
%
% Outputs:
%         db/ds
%
%-----%

beta2 = [db_ds2; db_ds3]/delq;

dz(1) = z(2);

dz(2) = -(D2*beta1 + C(1,:)*dq' + G(1)) / (D1 + D2*beta2);

dz = [dz(1), dz(2)]';
end

```

2 Optimize file

```
    clc; clear;
% Optimize intial conditions for ZD states and Bezier coeffs using fmincon
%
% Zero dynamics simulator takes pre-impact condition so that should be I.C
%
% for fmincon as well

set_path

%-----%

%[q1, dq1] pre impact conditions - should be negative for both
z_minus = [-0.325, -3.0765];

% Bezier coefficients
%      3      6      9
alpha = [0.1520, 0.1604, 0.7711];           %for q2 - alpha 3rd - 5th
%      120    90     60
gamma = [2.8535, 2.8806, 3.2398];           %for q3 - alpha 3rd - 5th

%   f0 = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
%   q10: pre-impact initial angle for q1
%   dq10: pre-impact initial velocity for dq1
%   alpha(3-5)_q2:
%           3rd to 5th Bezier coefficient for q2
%   alpha(3-5)_q3:
%           3rd to 5th Bezier coefficient for q3
%
f0 = [z_minus, alpha, gamma]; %parameters that need to be optimized

%-----%

% Lower and upper bounds for optimizer - [q1 q1_dot, alpha q2, alpha q3]
%
% epsilon for angles
eps_alpha = 0.05; %tighter window for leg angles

eps_gamma = 0.35; % Wider window for torso angles

eps = 0.1 % for q1

% epsilon for velocity
eps_v = 1.0;

lb = [z_minus(1)-eps, z_minus(2)-eps_v, alpha-eps_alpha, gamma-eps_gamma];
ub = [z_minus(1)+eps, z_minus(2)+eps_v, alpha+eps_alpha, gamma+eps_gamma];

%-----%
```

```

%% Run optimizer

% Options:
opts = optimoptions('fmincon','Algorithm','interior-point',...
    'MaxIterations',1000,...
    'StepTolerance',1e-6,...
    'FunctionTolerance',1e-2);
opts.Display = 'iter';

% using only bounds and nonlinear constraints
%
% Outputs: f - optimized parameters [qt, dqt, qt_LR_max, alphaR, alphaL]
%          J - Cost to minimize
%
[f, J] = fmincon(@(f) pass_cost_ZD(f), f0, [],[],[],[], lb,ub, @confuneq,
    opts);

disp('solution found!');

f

% Function only exists to pass cost as a scalar output for fmincon, actual
% cost calculation happens in func_cost_ZD
%
function J = pass_cost_ZD(y)

[J, ~] = func_cost_ZD(y);

end

%-----%

% Simulate a single step of zero dynamics and get cost
%
% Inputs: [z_plus, a, s_params]
%         z_plus: [q1, dq1] post impact condition
%         a: [alpha2, alpha3] Bezier coefficients for q2 and q3
%         s_params: [q1_min, q1_max]
%
% Outputs: [J, z_sol]
%          J: Cost
%          z_sol: solution to zero dynamics ODE45
%
function [J, z_sol] = func_cost_ZD(f)

%% Simulate

% Simulation of a single step of ZD to using preimpact conditions
% applies impact first then simulates zero dynamics
%
% Inputs: f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
%          q10: pre-impact initial angle for q1
%          dq10: pre-impact initial velocity for dq1

```

```

%      alpha(3-5)_q2:
%          3rd to 5th Bezier coefficient for q2
%      alpha(3-5)_q3:
%          3rd to 5th Bezier coefficient for q3
%
% Outputs:
%      t_sol - time (s) of zero dynamics
%      z_sol - [q1, dq1] post impact dynamics
%
[t_sol, z_sol] = sim_zero_dynamics(f);

%%% Compute cost

% Variables needed to compute control action
q1_min = -f(1);
q1_max = f(1);

s_params = [q1_min, q1_max];

alpha2 = [-f(5)+2*f(3), -f(4)+2*f(3), f(3:5)];
alpha3 = [-f(8)+2*f(6), -f(7)+2*f(6), f(6:8)];

a = [alpha2, alpha3];

J = 0;

for i = 2:length(t_sol)

    % Post compute control action
    %
    % Inputs: [z, a, s_params]
    %      z: [q1, dq1]
    %      a: [alpha2, alpha3] - 1st to 5th Bezier coefficient for q2 and
    %          q3
    %      s_params: [q1_min, q1_max]
    %
    % Output:
    %      u: control action
    %
    u = func_compute_control_action(z_sol(i,:),a,s_params);

    dt = t_sol(i) - t_sol(i-1);

    % Integral of squared control effort
    J = J + norm(u)^2 * dt;

end

z_final = z_sol(end,:);

% Compute the final angle and velocity constraints
q1_final = z_final(1);
dq1_final = z_final(2);

```

```

%Compute joint positions at end of swing
q = [q1_final, alpha2(end),alpha3(end)];
dq = [dq1_final,0,0];

%Compute model parameters
[r,m,Mh,Mt,l,g] = func_model_params;
params = [r,m,Mh,Mt,l,g];

%Compute P2H at end of swing leg
[~,~,~,~,~,P2] = func_compute_pMh_pMt_pm1_pm2_pcm_P2(q,dq,params);

% Divide by step length

% Horizontal component of P2 (end of swing leg)
step_length = abs(P2(1));

% Normalize cost by step length
if step_length < 1e-6 || isnan(J) || isinf(J)
    J = 1e6;
else
    J = J / step_length;
end

end

%-----%

% Nonlinear constraint for optimizer
%
% Inputs: f: [q1, dq1, alpha2(3:5), alpha3(3:5)]
%
% Output: [c, ceq]
%         c: nonlinear inequality constraint
%         ceq: nonlinear equality constraint
%
function [c,ceq] = confuneq(f)

z_i = f(1:2);          %Pre-impact states

% Simulation of a single step of ZD to using preimpact conditions
%   applies impact first then simulates zero dynamics
%
% Inputs: f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
%          q10: pre-impact initial angle for q1
%          dq10: pre-impact initial velocity for dq1
%          alpha(3-5)_q2:
%                   3rd to 5th Bezier coefficient for q2
%          alpha(3-5)_q3:
%                   3rd to 5th Bezier coefficient for q3
%
% Outputs:
%          t_sol - time (s) of zero dynamics
%          z_sol - [q1, dq1] post impact dynamics
%

```



```

[~, z_sol] = sim_zero_dynamics(f);

z_f = z_sol(end,:);      %final values after swing phase and right before
    the next impact

% Nonlinear inequality constraints
c = [0.3 - abs(f(1))];   % require |q1| >= 0.15 rad

% Nonlinear equality constraints
ceq = [z_i(1) - z_f(1);
       z_i(2) - z_f(2)];

% Force pre-impact condition and final solution of ODE45 to be the same

end

```

3 Simulate and Plot Zero Dynamics

```

% Include util and autogen folders
set_path

% f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
% q10: pre-impact initial angle for q1
% dq10: pre-impact initial velocity for dq1
% alpha(3-5)_q2:
%           3rd to 5th Bezier coefficient for q2
% alpha(3-5)_q3:
%           3rd to 5th Bezier coefficient for q3
%
% Manually tuned (working):
f = [ -0.64896      -2.2698      0.10266      0.21016      0.72138
      3.0566       3.0742       3.0025];

z_tot = [];

n = 100;

for i = 1:n

    % Simulation of a single step of ZD to using preimpact conditions
    % applies impact first then simulates zero dynamics
    %
    % Inputs: f = [q10, dq10, alpha(3-5)_q2, alpha(3-5)_q3]
    % q10: pre-impact initial angle for q1
    % dq10: pre-impact initial velocity for dq1
    % alpha(3-5)_q2:
    %           3rd to 5th Bezier coefficient for q2
    % alpha(3-5)_q3:
    %           3rd to 5th Bezier coefficient for q3
    %
    % Outputs:

```

```

%      t_sol - time (s) of zero dynamics
%      z_sol - [q1, dq1] post impact dynamics
%
[t_sol, z_sol] = sim_zero_dynamics(f);

f(1:2) = z_sol(end,:);

z_tot = [z_tot; z_sol];

end

figure
plot(z_tot(1,1),z_tot(1,2),'x'), hold on
plot(z_tot(:,1),z_tot(:,2))
hold off, grid on
title('Phase portrait of q_1 vd dq_1')
xlabel('q_1 (rads)')
ylabel('dq_1 (rads/s)')
legend('Start','Trajectory')

```

4 Results

Initial guess:

$$f_0 = [q_1, \dot{q}_1, \alpha_3, \alpha_4, \alpha_5, \gamma_3, \gamma_4, \gamma_5] \quad (1)$$

$$f_0 = [-0.325, -3.0765, 0.1520, 0.1604, 0.7711, 2.8535, 2.8806, 3.2398] \quad (2)$$

The vector f_0 represents the initial guess for the optimization variables, composed of the pre-impact conditions $(q_1, \dot{q}_1)$ and the last three Bezier coefficients of α and γ .

Optimized solution found by the optimizer:

$$f = [-0.64896, -2.2698, 0.10266, 0.21016, 0.72138, 3.0566, 3.0742, 3.0025] \quad (3)$$

The vector f represents the optimized solution obtained from the optimization process.

After adding the optimized solution to the Simulate and Plot Zero Dynamics file the following Plot was obtained.